一. Eigen 库

官网：http://eigen.tuxfamily.org。
文档：http://eigen.tuxfamily.org/dox/。
快速参考：http://eigen.tuxfamily.org/dox/group__QuickRefPage.html。

二. Eigen 基本操作

完整代码：https://github.com/deng-ke/slam-base/tree/master/EigenMatrix。

2.1 模块和头文件

1	Core	#include<Eigen/Core>	Matrix/Array 类、基础运算
2	Geometry	#include<Eigen/Geometry>	旋转/平移/缩放，2D/3D 变换
3	LU	#include<Eigen/LU>	求逆，行列式，LU 分解
4	Cholesky	#include<Eigen/Cholesky>	LLT 和 LDLT Cholesky 分解
5	Householder	#include<Eigen/Householder>	Householder 变换
6	SVD	#include<Eigen/SVD>	SVD 分解
7	QR	#include<Eigen/QR>	QR 分解
8	Eigenvalues	#include<Eigen/Eigenvalues>	特征值/特征向量分解
9	Sparse	#include<Eigen/Sparse>	稀疏矩阵的存储和运算
10	Dense	#include<Eigen/Dense>	包含了 1-4、6-8 头文件
11	Eigen	#include<Eigen/Eigen>	Dense/Sparse 头文件

2.2 Matrix

2.2.1 矩阵、向量定义

- **基本定义：**

矩阵、向量都是 Matrix 模板类的对象，主要使用前三个参数，剩下为默认值。

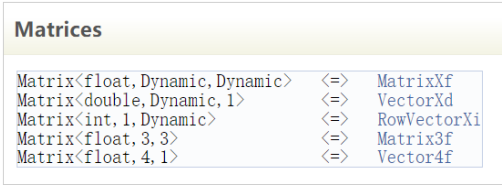
```
Eigen::Matrix<float, 2, 3> matrix_23;
Eigen::Matrix<float,3,1> vd_3d;
```

- **动态大小：**

```
Eigen::Matrix< double, Eigen::Dynamic, Eigen::Dynamic > matrix_dynamic1;
Eigen::Matrix< double, 1, Eigen::Dynamic > matrix_dynamic2;
```

- **简写定义：**

  typedef 了一些简写：




```
Eigen::Vector3d v_3d;
Eigen::MatrixXd matrix_x;
```

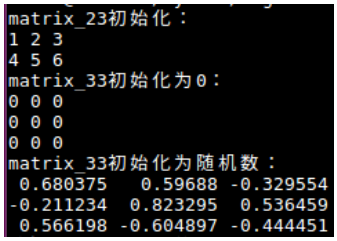
2.2.2 矩阵、向量初始化

- **直接初始化：**

```
matrix_23 << 1, 2, 3, 4, 5, 6;
```

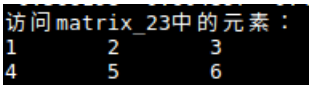
- **初始化为 0：**

```
Eigen::Matrix3d matrix_33 = Eigen::Matrix3d::Zero();
```



2.2.3 访问矩阵中的元素

```
for (int i=0; i<2; i++) {
    for (int j=0; j<3; j++)
        cout<<matrix_23(i,j)<<"\t";
    cout<<endl;
}
```



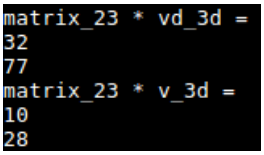
2.3 运算

- **四则运算：**

  以矩阵和向量相乘为例：

```
v_3d << 3, 2, 1;
vd_3d << 4,5,6;
Eigen::Matrix<float, 2, 1> result1 = matrix_23 * vd_3d;
```

```
cout << "matrix_23 * vd_3d = " << endl << result1 << endl;
Eigen::Matrix<double, 2, 1> result2 = matrix_23.cast<double>() * v_3d;
cout << "matrix_23 * v_3d = " << endl << result2 << endl;   //需显式转换类型
```



- **其它运算：**

  包括转置、逆、迹等：

```
cout << "转置：" << endl << matrix_33.transpose() << endl;
cout << "各元素和：" << endl << matrix_33.sum() << endl;
cout << "迹：" << endl << matrix_33.trace() << endl;
cout << "数乘：" << endl << 10*matrix_33 << endl;
cout << "逆：" << endl << matrix_33.inverse() << endl;
cout << "行列式：" << endl << matrix_33.determinant() << endl;
//实对称矩阵可保证对角化成功：
Eigen::SelfAdjointEigenSolver<Eigen::Matrix3d>                                    eigen_solver
( matrix_33.transpose()*matrix_33 );
cout << "特征值和特征向量：" << endl;
cout << "Eigen values = \n" << eigen_solver.eigenvalues() << endl;
cout << "Eigen vectors = \n" << eigen_solver.eigenvectors() << endl;
```



2.4 解方程

两种方法：直接求逆、利用 QR 分解。方法 2 更快。

- **直接求逆解方程：**

```
clock_t time_stt = clock();
Eigen::Matrix<double,MATRIX_SIZE,1> x = matrix_NN.inverse()*v_Nd;
cout << "直接求逆解方程：" << endl;
cout << "time use=" << endl << 1000* (clock() - time_stt)/(double)CLOCKS_PER_SEC << "ms"<< endl;
cout << "x=" << endl << x << endl;
```

```
直接求逆解方程：
time use=
0.247ms
x=
   -0.683262
    -1.02484
   0.0528991
   -0.968925
    0.234639
 -0.00613843
    0.663124
   -0.444261
    0.321786
    0.271768
-0.000526314
   -0.211471
     1.48623
     0.77974
    0.280403
    -1.43433
     1.27476
    -1.92547
   -0.317352
   -0.246389
```

- **用 QR 分解解方程：**

```
time_stt = clock();
x = matrix_NN.colPivHouseholderQr().solve(v_Nd);
cout << "利用 QR 分解解方程：" << endl;
cout << "time use=" << endl << 1000*    (clock() - time_stt)/(double)CLOCKS_PER_SEC <<"ms" << endl;
cout << "x=" << endl << x << endl;
```

```
利用QR分解解方程：
time use=
0.025ms
x=
   -0.683262
    -1.02484
   0.0528991
   -0.968925
    0.234639
 -0.00613843
    0.663124
   -0.444261
    0.321786
    0.271768
-0.000526314
   -0.211471
     1.48623
     0.77974
    0.280403
    -1.43433
     1.27476
    -1.92547
   -0.317352
   -0.246389
```

三. Eigen 实现位姿表示

完整代码：https://github.com/deng-ke/slam-base/tree/master/EigenGeometry。

主要用到 Geometry 模块，该模块实现的类，包含旋转矩阵、变换矩阵、旋转向量、欧拉角、四元数等对位姿的描述方式。

class	**Eigen::AlignedBox< _Scalar, _AmbientDim >** An axis aligned box. More...
class	**Eigen::AngleAxis< _Scalar >** Represents a 3D rotation as a rotation angle around an arbitrary 3D axis. More...
class	**Eigen::Homogeneous< MatrixType, _Direction >** Expression of one (or a set of) homogeneous vector(s) More...
class	**Eigen::Hyperplane< _Scalar, _AmbientDim, _Options >** A hyperplane. More...
class	**Eigen::Map< const Quaternion< _Scalar >, _Options >** **Quaternion** expression mapping a constant memory buffer. More...
class	**Eigen::Map< Quaternion< _Scalar >, _Options >** Expression of a quaternion from a memory buffer. More...
class	**Eigen::ParametrizedLine< _Scalar, _AmbientDim, _Options >** A parametrized line. More...
class	**Eigen::Quaternion< _Scalar, _Options >** The quaternion class used to represent 3D orientations and rotations. More...
class	**Eigen::QuaternionBase< Derived >** Base class for quaternion expressions. More...
class	**Eigen::Rotation2D< _Scalar >** Represents a rotation/orientation in a 2 dimensional space. More...
class	**Scaling** Represents a generic uniform scaling transformation. More...
class	**Eigen::Transform< _Scalar, _Dim, _Mode, _Options >** Represents an homogeneous transformation in a N dimensional space. More...
class	**Eigen::Translation< _Scalar, _Dim >** Represents a translation transformation. More...

3.1 旋转向量/矩阵

AngleAxis 定义旋转向量；Matrix3 定义旋转矩阵。两种方式可将旋转向量转换为旋转矩阵。

```
/*****************************************************************************
 *旋转向量/矩阵；旋转向量转换为旋转矩阵
 *****************************************************************************/
    Eigen::AngleAxisd rotation_vector ( M_PI/4, Eigen::Vector3d ( 0,0,1 ) );
    Eigen::Matrix3d rotation_matrix = Eigen::Matrix3d::Identity();
    Eigen::Vector3d v ( 1,0,0 );
    cout .precision(3);
    /*两种方式：旋转向量 to 旋转矩阵*/
    cout<<"rotation matrix =\n"<<rotation_vector.matrix() <<endl;
    rotation_matrix = rotation_vector.toRotationMatrix();
```

```
/*使用旋转向量变换*/
Eigen::Vector3d v_rotated = rotation_vector * v;
cout<<"(1,0,0) after rotation = "<<v_rotated.transpose()<<endl;
/*使用旋转矩阵变换*/
v_rotated = rotation_matrix * v;
cout<<"(1,0,0) after rotation = "<<v_rotated.transpose()<<endl<<endl;
```

```
rotation matrix =
 0.707 -0.707      0
 0.707  0.707      0
     0      0      1
(1,0,0) after rotation = 0.707 0.707     0
(1,0,0) after rotation = 0.707 0.707     0
```

3.2 变换矩阵

```
/*****************************************************************************
 *变换矩阵
 *****************************************************************************/
    /*定义，并用旋转矩阵、平移向量进行初始化*/
    //T 为 4*4。用于一个旋转 3 维向量
    Eigen::Isometry3d T=Eigen::Isometry3d::Identity();
    T.rotate ( rotation_vector );
    T.pretranslate ( Eigen::Vector3d ( 1,3,4 ) );
    cout << "Transform matrix = \n" << T.matrix() <<endl;
    /*使用变换矩阵*/
    Eigen::Vector3d v_transformed = T*v;
    cout<<"v tranformed = "<<v_transformed.transpose()<<endl<<endl;
```

```
Transform matrix =
 0.707 -0.707      0      1
 0.707  0.707      0      3
     0      0      1      4
     0      0      0      1
v tranformed = 1.71 3.71    4
```

3.3 欧拉角

```
/*****************************************************************************
 *欧拉角；
 *****************************************************************************/
    Eigen::Vector3d euler_angles = rotation_matrix.eulerAngles ( 2,1,0 );
    cout<<"yaw pitch roll = "<<euler_angles.transpose()<<endl<<endl;
```

```
yaw pitch roll = 0.785    -0     0
```

3.4 四元数

```
/*****************************************************************************
 *四元数
 *****************************************************************************/
    /*旋转矩阵转换为四元数*/
    Eigen::Quaterniond q = Eigen::Quaterniond ( rotation_vector );
    cout<<"quaternion = \n"<<q.coeffs() <<endl;
    /*旋转向量转换为四元数*/
    q = Eigen::Quaterniond ( rotation_matrix );
    cout<<"quaternion = \n"<<q.coeffs() <<endl;
    /*使用四元数*/
    v_rotated = q*v; //*经过了重载，原本：qvq-1
    cout<<"(1,0,0) after rotation = "<<v_rotated.transpose()<<endl;
```

```
quaternion =
    0
    0
0.383
0.924
quaternion =
    0
    0
0.383
0.924
(1,0,0) after rotation = 0.707 0.707     0
```